1)

```
module dynamic_array;
  int dyn_arr1[];
  int dyn_arr2[] = '{9,1,8,3,4,4};  // initializ

  initial
    begin
      dyn_arr1 = new[6];     // allocate six ele

      foreach (dyn_arr1[i])
          dyn_arr1[i] = i;

      $display(dyn_arr1,",",dyn_arr1.size);
      dyn_arr1.delete();

      dyn_arr2.reverse();            // reverse
      $display(dyn_arr2);

      dyn_arr2.sort();               // sorting
      $display(dyn_arr2);

      dyn_arr2.rsort();              // reverse
      $display(dyn_arr2);

      dyn_arr2.shuffle();            // shufflin
      $display(dyn_arr2);
  end
endmodule
```

2) The design has no bugs, and no exclusions needed.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	A checker in the testbench to make sure the output is correct
COUNTER_2	When the load is asserted, the output count_out should take the value of the load_data input	Randomization under constraints on the load signal to be off 70% of the time	-	A checker in the testbench to make sure the output is correct
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is high	Randomization for up_down, and randomization for enable to be 70% of the time	-	A checker in the testbench to make sure the output is correct
COUNTER_4	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is low	Randomization for up_down, and randomization for enable to be 70% of the time	-	A checker in the testbench to make sure the output is correct

Kareem Waseem

```
package counter_pkg;
parameter W = 4;

class counter_transaction;
rand bit rst_n, load_n, up_down, ce;
rand bit [W-1:0] data_load, count_out;

// COUNTER_1
constraint rst_con {
 rst_n dist {0:= 2, 1:= 98};
}

// COUNTER_2
constraint load_con {
 load_n dist {0:= 70, 1:= 30};
}

// COUNTER_3, COUNTER_4
constraint ce_con {
 ce dist {1:= 70, 0:= 30};
}
endclass
endpackage
```

```
import counter_pkg::*;

module counter_tb;
bit clk ,rst_n, load_n, up_down, ce;
bit [W-1:0] data_load;
logic max_count, max_count_exp, zero, zero_exp;
logic [W-1:0] count_out, count_out_exp;

counter_transaction c_txn = new;

int error_count;
int correct_count;

counter #(.WIDTH(W)) c1(clk ,rst_n, load_n, up_d

// Create the clock
initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Stimulus Generation
initial begin
   assert_reset;
```

Kareem Waseem

```
    repeat(10000) begin
        assert(c_txn.randomize());
        rst_n = c_txn.rst_n;
        load_n = c_txn.load_n;
        up_down = c_txn.up_down;
        ce = c_txn.ce;
        data_load = c_txn.data_load;
        check_result(c_txn);
    end

    $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
    $stop;
end

// Checker
task check_result(input counter_transaction c_txn_c);
    golden_model(c_txn_c);
    if (c_txn_c.rst_n !== 0) begin
        @(negedge clk);
        if ( (max_count_exp !== max_count) || (zero_exp !== zero) || (count_out_exp !== count_out) ) begin
            error_count = error_count + 1;
            $display("%t: Error: For rst_n=0d%0d, load_n= 0d%0d, up_down=0d%0d, ce= 0d%0d, data_load=0d%0d, count_out
        end
        else
            correct_count = correct_count + 1;
    end
    else begin //rst check
        check_rst;
    end

endtask
```

```
// Golden Model
task golden_model(input counter_transaction c_txn_g);
    if (!c_txn_g.rst_n)
        count_out_exp = 0;
    else if (!c_txn_g.load_n)
        count_out_exp = c_txn_g.data_load;
    else if (c_txn_g.ce)
            if (c_txn_g.up_down)
                count_out_exp = count_out_exp + 1;
            else
                count_out_exp = count_out_exp - 1;
    max_count_exp = (count_out_exp == {W{1'b1}})? 1:0;
    zero_exp = (count_out_exp == 0)? 1:0;
endtask

task assert_reset;
    //assert reset
    rst_n = 0;
    check_rst;
    //desert reset
    rst_n = 1;
endtask

task check_rst;
    @(negedge clk);
    if (count_out !== 0 && max_count !== 0 && zero == 1) begin
        error_count = error_count + 1;
        $display("%t: Error: Reset value is asserted, outputs are with wrong reset values", $time);
    end
    else
        correct_count = correct_count + 1;
endtask
endmodule
```

Hareem Waseem

3) Bugs:

1. FULL_ADDER parameter implementation is not complete, when opcode is 3'h2.
2. Case statement is using opcode signal instead of opcode_reg

Exclusions: opcode case items missing default, and most sig bit toggle for cin_reg

Label	Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When the reset is asserted, the outputs should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases do not occur, when opcode is add, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	-	Output Checked against golden model
ALSU_3	Incase of invalid cases do not occur, when opcode is mult, then out should perform the multiplication on ports A and B	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	-	Output Checked against golden model
ALSU_4	Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B inputs are low	Randomization without constraints on A or B. reduction ports are disabled most of the time	-	Output Checked against golden model
ALSU_5	Incase of invalid cases do not occur, when opcode is OR & any of the inputs reduction_A or reduction_B is high, then out should perform the reduction OR operation on ports A and B based on the priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high.	-	Output Checked against golden model
ALSU_6	Incase of invalid cases do not occur, when opcode is XOR, then out should perform the XOR operation on ports A and B if reduction_A and reduction_B inputs are low	Randomization without constraints on A or B. reduction ports are disabled most of the time	-	Output Checked against golden model
ALSU_7	Incase of invalid cases do not occur, when opcode is XOR & any of the inputs reduction_A or reduction_B is high, then out should perform the reduction XOR operation on ports A and B based on the priority	Randomization under constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	-	Output Checked against golden model
ALSU_8	Incase of invalid cases do not occur, when opcode is SHIFT then the output out will shift right or left based on the direction input	Randomization without constraints	-	Output Checked against golden model
ALSU_9	Incase of invalid cases do not occur, when opcode is ROTATE then the output out will rotate right or left based on the direction input	Randomization without constraints	-	Output Checked against golden model
ALSU_10	When invalid cases exist without bypass, out output should be low and leds should blink	Randomization under constraints where invalid cases do not occur as frequent as valid cases	-	Output Checked against golden model
ALSU_11	If invalid cases do not occur and the bypass inputs are high, then the output out should by bypass port A or B based on the prioirty if the both bypass ports are high	Randomization under constraint on bypass ports to be disabled most of the time	-	Output Checked against golden model

Karim Yasen

```
package ALSU_pkg;
typedef enum bit [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
typedef enum bit [2:0] {MAXPOS=3'b011, ZERO = 3'b000, MAXNEG=3'b100} reg_e;

class ALSU_inputs;
rand logic clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
rand logic [2:0] A, B;
rand opcode_e opcode;
rand reg_e enum_ext_A, enum_ext_B;
rand logic  [3:0] A_rem_values, B_rem_values;
bit [2:0] walking_ones[] = '{3'b001, 3'b010, 3'b100};
rand bit [2:0] walking_ones_t, walking_ones_f;

// ALSU_1
constraint rst_con {
 rst dist {1:= 2, 0:= 98};
}

constraint A_B_con {
   A_rem_values != MAXPOS || 0 || MAXNEG;
   B_rem_values != MAXPOS || 0 || MAXNEG;
   walking_ones_t inside {walking_ones};
   !(walking_ones_f inside {walking_ones});

   if (opcode == OR || opcode == XOR) {
      // ALSU_5, ALSU_7
      if (red_op_A == 1) {
         B == 0;
         A dist {walking_ones_t:= 80, walking_ones_f:= 20};
      }  else if (red_op_B == 1) {
         A == 0;
         B dist {walking_ones_t:= 80, walking_ones_f:= 20};
      }
   } else {
      // ALSU_4, ALSU_6
      red_op_A dist {1:= 20, 0:= 80};
      red_op_B dist {1:= 20, 0:= 80};
      // ALSU_2, ALSU_3
      if (opcode == ADD || opcode == MULT) {
         A dist {enum_ext_A:= 80, A_rem_values:= 20};
         B dist {enum_ext_B:= 80, B_rem_values:= 20};
      }
   }
}
```

```
// ALSU_10
constraint opcode_con {
   opcode dist {[OR:ROTATE]:= 80, [INVALID_6:INVALID_7]:= 20};
}

// ALSU_11
constraint bypass_con {
   bypass_A dist {1:= 2, 0:= 98};
   bypass_B dist {1:= 2, 0:= 98};
}

endclass
endpackage
```

Testbench with golden model

```
import ALSU_pkg::*;

module ALSU_tb;
  parameter INPUT_PRIORITY = "A" ;
  parameter FULL_ADDER = "ON";
  bit clk ,rst ,cin ,red_op_A ,red_op_B ,bypass_A ,bypass_B ,direction ,serial_in;
  bit signed [2:0] A , B;
  opcode_e opcode;
  bit [15:0] leds_exp;
  logic [15:0] leds;
  logic signed [5:0] out;
  bit signed [5:0] out_exp;

  //Internals to register the stimulus for golden model
  bit red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
  bit signed [1:0] cin_reg;
  bit [2:0] opcode_reg;
  bit signed [2:0] A_reg, B_reg;

  int error_count = 0;
  int correct_count = 0;

  ALSU_inputs ALSU_obj = new();

  ALSU a1 (A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, 

  // clock Generation
  initial begin
    clk = 0 ;
    forever
    #1 clk = ~clk ;
  end

  initial begin
    assert_reset();
    // start randomization
    repeat (20000)begin
      assert(ALSU_obj.randomize());
      cin = ALSU_obj.cin;
      red_op_A = ALSU_obj.red_op_A;
      red_op_B = ALSU_obj.red_op_B ;
      bypass_A = ALSU_obj.bypass_A;
      bypass_B = ALSU_obj.bypass_B;
      direction = ALSU_obj.direction;
      serial_in = ALSU_obj.serial_in;
      opcode = ALSU_obj.opcode;
      A = ALSU_obj.A;
      B = ALSU_obj.B;
      rst = ALSU_obj.rst;
      if (ALSU_obj.rst) begin
        check_rst();
        reset_internals();
      end
```

```
      else begin
        golden_model();
        check_results();
      end
    end
    $display("correct_count = %0d and error_count =%0d",correct_count , error_count);
    $stop ;
  end

  task assert_reset();
    // reset asserted
    rst = 1;
    @(negedge clk);
    check_rst();
    rst = 0;
  endtask

  task check_rst();
    @(negedge clk);
    if (out != 0 || leds != 0) begin
      $display("Error in reset");
      error_count++;
    end
    else
      correct_count++;
    reset_internals();
  endtask

  task reset_internals();
    {red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg} = 6'b00_0000;
    cin_reg = 2'b00;
    opcode_reg = 3'b00;
    {A_reg, B_reg} = 2'b00;
  endtask

  // check results task
  task check_results();
    @(negedge clk);
      if ((out == out_exp) && (leds == leds_exp)) begin
        correct_count++;
      end
      else begin
        error_count++;
        $display("%0t : error when A =0d'%0d , B =0d'%0d , cin =0d'%0d, serial_in =0d'%0d, red_op_A =0d'%0d, re
        $display("leds =16'h%0h, leds_exp = 6'h%0h and  out =0d'%0d , out_exp = 0d'%0d ",leds , leds_exp , out
        $stop;
      end
   // end
  endtask
```

```
function bit is_invalid();
  //invalid case 1
  if (opcode_reg == INVALID_6 || opcode_reg == INVALID_7)
    return 1;
  //invalid case 2
  else if ((opcode_reg > 3'b001) && (red_op_A_reg || red_op_B_reg))
    return 1;
  else
    return 0;
endfunction

task golden_model();
  //Handle leds_exp
  if(is_invalid())
    leds_exp = ~leds_exp;
  else
    leds_exp = 0;

  //Handle bypass, invalid and valid opcodes for out_exp

  // Check Bypass
  if (bypass_A_reg)
    out_exp = A_reg;
  else if (bypass_B_reg)
    out_exp = B_reg;

  //Check invalid
  else if (is_invalid())
    out_exp = 0;

  //Valid opcodes
  else begin
    // OR
    if (opcode_reg == OR) begin
      if (red_op_A_reg)
          out_exp = |A_reg;
        else if (red_op_B_reg)
          out_exp = |B_reg;
        else
          out_exp = A_reg | B_reg;
    end

    // XOR
    else if (opcode_reg == XOR) begin
      if (red_op_A_reg)
          out_exp = ^A_reg;
        else if (red_op_B_reg)
          out_exp = ^B_reg;
        else
          out_exp = A_reg ^ B_reg;
    end
```

Karem Waseem

```
      // ADD
      else if (opcode_reg == ADD)
        out_exp = A_reg + B_reg + cin_reg;

      // MULT
      else if (opcode_reg == MULT)
        out_exp = A_reg * B_reg;

      // SHIFT
      else if (opcode_reg == SHIFT) begin
        if (direction_reg)
            out_exp = {out_exp[4:0], serial_in_reg};
          else
            out_exp = {serial_in_reg , out_exp[5:1]};
      end

      // ROTATE
      else if (opcode_reg == ROTATE) begin
        if (direction_reg)
            out_exp = {out_exp[4:0], out_exp[5]};
          else
            out_exp = {out_exp[0], out_exp[5:1]};
      end
    end
    update_internals();
  endtask

  task update_internals();
    cin_reg = cin;
    red_op_B_reg = red_op_B;
    red_op_A_reg = red_op_A;
    bypass_B_reg = bypass_B;
    bypass_A_reg = bypass_A;
    direction_reg = direction;
    serial_in_reg = serial_in;
    opcode_reg = opcode;
    A_reg = A;
    B_reg = B;
  endtask

endmodule
```

Kareem Waseem

OR use a testbench with instantiation

```
import ALSU_pkg::*;

module ALSU_tb;
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";
bit clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
bit [2:0] A, B;
opcode_e opcode;
logic [15:0] leds;
bit [15:0] leds_exp;
logic [5:0] out;
bit [5:0] out_exp;

int error_count;
int correct_count;

ALSU_inputs ALSU_obj = new();

ALSU a1 (A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direct
ALSU ref1 (A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, dire

// Create the clock
initial begin
   clk= 0;
   forever
      #1 clk= ~clk;
end

initial begin
   assert_reset();

   repeat (10000) begin
      assert(ALSU_obj.randomize());
      cin = ALSU_obj.cin;
      red_op_A = ALSU_obj.red_op_A;
      red_op_B = ALSU_obj.red_op_B;
      bypass_A = ALSU_obj.bypass_A;
      bypass_B = ALSU_obj.bypass_B;
      direction = ALSU_obj.direction;
      serial_in = ALSU_obj.serial_in;
      opcode = ALSU_obj.opcode;
      A =  ALSU_obj.A;
      B = ALSU_obj.B;
      rst = ALSU_obj.rst;
      check_result(ALSU_obj);
   end

   $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count
   $stop;
end
```

Kareem Waseem

```
task check_result(input ALSU_inputs ALSU_obj_c);
   if (ALSU_obj_c.rst !== 1) begin
      @(negedge clk);
      if ( (out !== out_exp) || (leds !== leds_exp)) begin
        error_count = error_count + 1;
        $display("%t: Error: For A=0d%0d, B= 0d%0d, cin=0d%0d,
      end
      else
        correct_count = correct_count + 1;
   end
   else begin //rst check
      check_rst;
   end
endtask

task assert_reset;
   //assert reset
   rst = 1;
   check_rst;
   //desert reset
   rst = 0;
endtask

task check_rst;
   @(negedge clk);
   if (out !== 0 && leds !== 0) begin
     error_count = error_count + 1;
     $display("%t: Error: Reset value is asserted, outputs are
   end
   else
     correct_count = correct_count + 1;
endtask

endmodule
```

Kareem Waseem